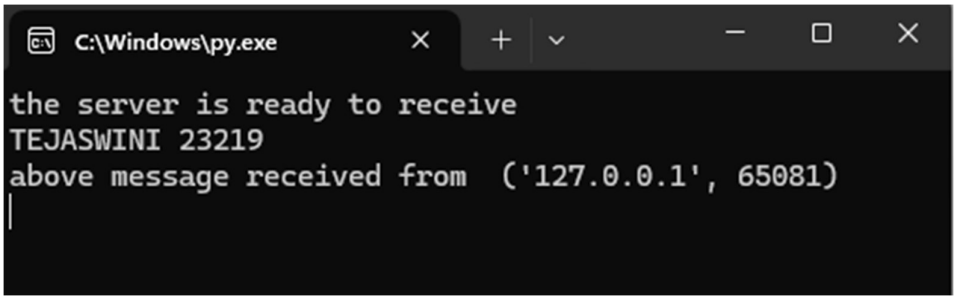
Computer Networks Labsheet 4

Socket Programming

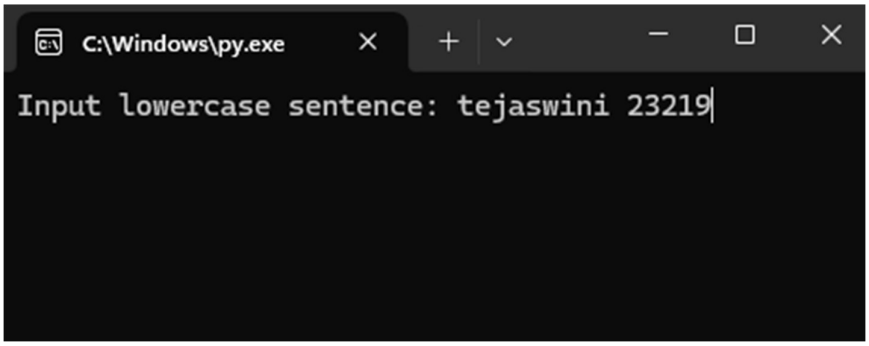
Part1: Understanding the UDP socket programming

1. Understand the existing UDP client and server Python program used to send messages among each other. Change the port number if required.

Server:



Client:

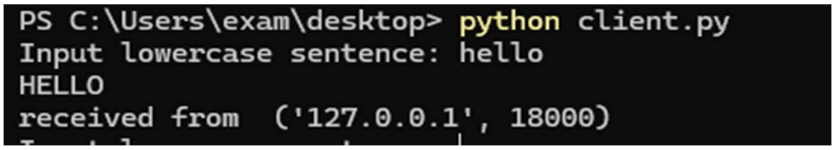


2. Execute the client and server code simultaneously. Have the screenshot in

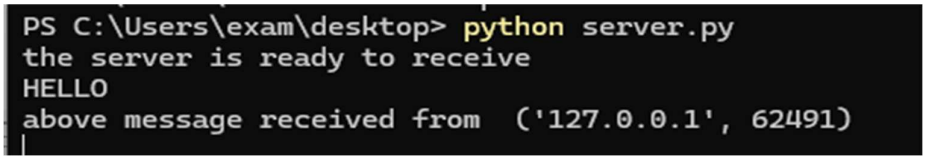
such a way that the command prompt captures your name and roll number.

Sample screenshots:

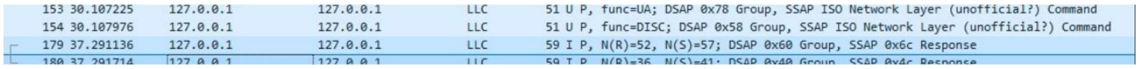
@client



@server



3. Try to capture the wireshark frames filtering for udp captured in the loopback interface is shown below. Please make sure your identity is visible. You can use AUMS login. Always follow this protocol in all your lab submissions in this course



4. Try to change the local host of client/server host to remote host and use right interface to capture the messages in wireshark.

```
PS C:\Users\exam\desktop> python client.py
Input lowercase sentence: tejawini
TEJAWINI
received from  ('127.0.0.1', 12000)
Input lowercase sentence: 23219
23219
received from  ('127.0.0.1', 12000)
Input lowercase sentence: bye
BYE
received from  ('127.0.0.1', 12000)
PS C:\Users\exam\desktop>
```

```
PS C:\Users\exam\desktop> python server.py
the server is ready to receive
TEJAWINI
above message received from  ('127.0.0.1', 53375)
23219
above message received from  ('127.0.0.1', 53376)
BYE
above message received from  ('127.0.0.1', 64808)
```

Part2: Understanding the TCP socket programming

1. Understand the existing TCP client and server Python program used to send messages among each other. Change the port number if required.

TCP Client:

```
PS C:\Users\exam\desktop> python clientTCP.py
Input lowercase sentence:hello
From Server: HELLO
```

Server:

```
PS C:\Users\exam\desktop> python serverTCP.py
The server is ready to receive
sending  HELLO
to  ('127.0.0.1', 18263)
```

2. Execute the client and server code simultaneously in the same local host. Have the screenshot in such a way that the command prompt captures your name and roll number

Server:

```
PS C:\Users\garla\Downloads> python serverTCP.py
The server is ready to receive
sending  TEJASWINI 23219
to  ('127.0.0.1', 62236)
sending  AMRITA VSHWA VIDYAPEETHAM
to  ('127.0.0.1', 62238)
sending  AMMA
to  ('127.0.0.1', 64746)
```

Client:

```
PS C:\Users\garla\Downloads> python clientTCP.py
Input lowercase sentence:tejaswini 23219
From Server: TEJASWINI 23219
```

```
PS C:\Users\garla\Downloads> python clientTCP.py
Input lowercase sentence:amrita vshwa vidyapeetham
From Server: AMRITA VSHWA VIDYAPEETHAM
PS C:\Users\garla\Downloads> python clientTCP.py
Input lowercase sentence:amma
From Server: AMMA
PS C:\Users\garla\Downloads>
```

3. Try to capture the relevant wireshark frames and screenshot with proper identity rules specified earlier.

49	11.974291	127.0.0.1	127.0.0.1	TCP	63 20093 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
50	11.974337	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=1 Ack=20 Win=255 Len=0
51	11.975144	127.0.0.1	127.0.0.1	TCP	63 12456 → 20093 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=19
52	11.975177	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=20 Win=255 Len=0
53	11.975216	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [FIN, ACK] Seq=20 Ack=20 Win=255 Len=0
54	11.975234	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=21 Win=255 Len=0
55	11.975630	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [FIN, ACK] Seq=20 Ack=21 Win=255 Len=0
56	11.975694	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=21 Ack=21 Win=255 Len=0

69 13.403167	127.0.0.1	127.0.0.1	TCP	56 47500 → 12456 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
70 13.403247	127.0.0.1	127.0.0.1	TCP	56 12456 → 47500 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
71 13.403269	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=1 Ack=1 Win=65280 Len=0
96 20.806105	127.0.0.1	127.0.0.1	TCP	71 47500 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=27
97 20.806150	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [ACK] Seq=1 Ack=28 Win=65280 Len=0
98 20.807057	127.0.0.1	127.0.0.1	TCP	71 12456 → 47500 [PSH, ACK] Seq=1 Ack=28 Win=65280 Len=27
99 20.807093	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=28 Ack=28 Win=65280 Len=0
100 20.807129	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [FIN, ACK] Seq=28 Ack=28 Win=65280 Len=0
101 20.807146	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=28 Ack=29 Win=65280 Len=0
102 20.807329	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [FIN, ACK] Seq=28 Ack=29 Win=65280 Len=0
103 20.807382	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [ACK] Seq=29 Ack=29 Win=65280 Len=0

4. Execute at least one client and server process simultaneously in the different host. Have the screenshot in such a way that the command prompt captures your name and roll number.

Client:

```
PS C:\Users\garla\Downloads> python clientTCP.py
Input lowercase sentence:tejaswini 23219
From Server: TEJASWINI 23219
PS C:\Users\garla\Downloads> python clientTCP.py
Input lowercase sentence:computer networks
From Server: COMPUTER NETWORKS
```

Server:

```
sending  TEJASWINI 23219
to  ('127.0.0.1', 56959)
sending  COMPUTER NETWORKS
to  ('127.0.0.1', 56960)
```

5. Guess the frames used for TCP connection establishments before the data transfer. Provide the screenshots of 3 handshake frames(SYN, SYN ACK, ACK) in the list pane and identify the sender & receiver process for each of them.

49 11.974291	127.0.0.1	127.0.0.1	TCP	63 20093 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
50 11.974337	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=1 Ack=20 Win=255 Len=0
51 11.975144	127.0.0.1	127.0.0.1	TCP	63 12456 → 20093 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=19
52 11.975177	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=20 Win=255 Len=0
53 11.975216	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [FIN, ACK] Seq=20 Ack=20 Win=255 Len=0
54 11.975234	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=21 Win=255 Len=0
55 11.975630	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [FIN, ACK] Seq=20 Ack=21 Win=255 Len=0
56 11.975694	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=21 Ack=21 Win=255 Len=0

6. Guess the frames used for closing the TCP connection after the data transfer.

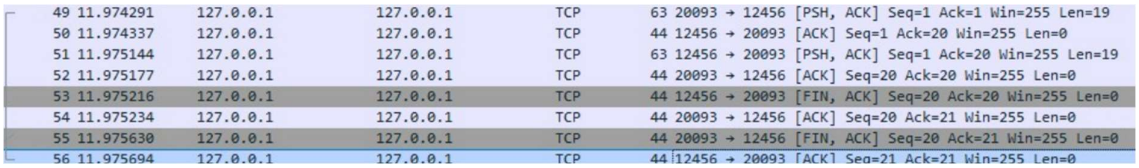
Provide the screenshots of frames (FIN ACK, ACK, FIN ACK, ACK) which

is involved in closing the connection.

69 13.403167	127.0.0.1	127.0.0.1	TCP	56 47500 → 12456 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
70 13.403247	127.0.0.1	127.0.0.1	TCP	56 12456 → 47500 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
71 13.403269	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=1 Ack=1 Win=65280 Len=0
96 20.806105	127.0.0.1	127.0.0.1	TCP	71 47500 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=27
97 20.806150	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [ACK] Seq=1 Ack=28 Win=65280 Len=0
98 20.807057	127.0.0.1	127.0.0.1	TCP	71 12456 → 47500 [PSH, ACK] Seq=1 Ack=28 Win=65280 Len=27
99 20.807093	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=28 Ack=28 Win=65280 Len=0
100 20.807129	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [FIN, ACK] Seq=28 Ack=28 Win=65280 Len=0
101 20.807146	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [ACK] Seq=28 Ack=29 Win=65280 Len=0
102 20.807329	127.0.0.1	127.0.0.1	TCP	44 47500 → 12456 [FIN, ACK] Seq=28 Ack=29 Win=65280 Len=0
103 20.807382	127.0.0.1	127.0.0.1	TCP	44 12456 → 47500 [ACK] Seq=29 Ack=29 Win=65280 Len=0

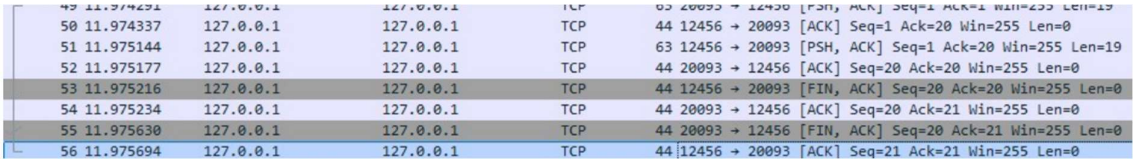
7. Try to capture the relevant wireshark frames and screenshot with proper

identity rules specified earlier.



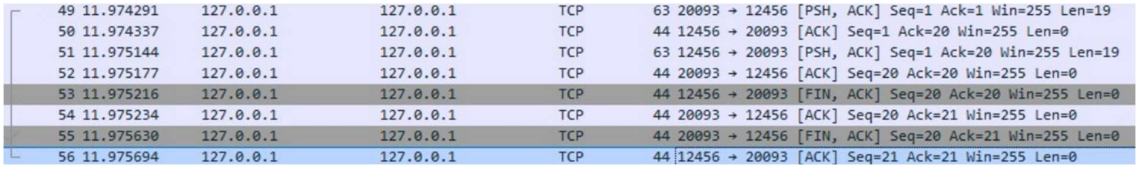
49	11.974291	127.0.0.1	127.0.0.1	TCP	63 20093 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
50	11.974337	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=1 Ack=20 Win=255 Len=0
51	11.975144	127.0.0.1	127.0.0.1	TCP	63 12456 → 20093 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=19
52	11.975177	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=20 Win=255 Len=0
53	11.975216	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [FIN, ACK] Seq=20 Ack=20 Win=255 Len=0
54	11.975234	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=21 Win=255 Len=0
55	11.975630	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [FIN, ACK] Seq=20 Ack=21 Win=255 Len=0
56	11.975694	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=21 Ack=21 Win=255 Len=0

8. Guess the frames used for TCP connection establishments before the data transfer. Provide the screenshots of 3 handshake frames(SYN, SYN ACK, ACK) in the list pane and identify the sender & receiver process for each of them.



50	11.974337	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=1 Ack=20 Win=255 Len=0
51	11.975144	127.0.0.1	127.0.0.1	TCP	63 12456 → 20093 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=19
52	11.975177	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=20 Win=255 Len=0
53	11.975216	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [FIN, ACK] Seq=20 Ack=20 Win=255 Len=0
54	11.975234	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=21 Win=255 Len=0
55	11.975630	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [FIN, ACK] Seq=20 Ack=21 Win=255 Len=0
56	11.975694	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=21 Ack=21 Win=255 Len=0

9. Guess the frames used for closing the TCP connection after the data transfer. Provide the screenshots of frames (FIN ACK, ACK, FIN ACK, ACK) which is involved in closing the connection.



49	11.974291	127.0.0.1	127.0.0.1	TCP	63 20093 → 12456 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=19
50	11.974337	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=1 Ack=20 Win=255 Len=0
51	11.975144	127.0.0.1	127.0.0.1	TCP	63 12456 → 20093 [PSH, ACK] Seq=1 Ack=20 Win=255 Len=19
52	11.975177	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=20 Win=255 Len=0
53	11.975216	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [FIN, ACK] Seq=20 Ack=20 Win=255 Len=0
54	11.975234	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [ACK] Seq=20 Ack=21 Win=255 Len=0
55	11.975630	127.0.0.1	127.0.0.1	TCP	44 20093 → 12456 [FIN, ACK] Seq=20 Ack=21 Win=255 Len=0
56	11.975694	127.0.0.1	127.0.0.1	TCP	44 12456 → 20093 [ACK] Seq=21 Ack=21 Win=255 Len=0

Part2: Webserver – application of TCP socket programming[Optional]

You will develop a web server that handles one HTTP request at a time. Your web server should accept and parse the HTTP request, get the requested file from the server's file system, create an HTTP response message consisting of the requested file preceded by header lines, and then send the response directly to the client. If the requested file is not present in the server, the server should send an HTTP "404 Not Found" message back to the client.

Code

Below you will find the skeleton code for the Web server. You are to complete the skeleton code. The places where you need to fill in code are marked with **#Fill in start** and **#Fill in end**. Each place may require one or more lines of code.

Running the Server Put an HTML file (e.g., HelloWorld.html) in the same directory that the server is in.

Run the server program. Determine the IP address of the host that is running the server (e.g., 128.238.251.26). From another host, open a browser and provide the corresponding URL. For example:

http://128.238.251.26:6789/HelloWorld.html

'HelloWorld.html' is the name of the file you placed in the server directory. Note also the use of the port number after the colon. You need to replace this port number with whatever port you have used in the server code. In the above example, we have used the port number 6789. The browser should then display the contents of HelloWorld.html. If you omit ":6789", the browser will assume port 80 and you will get the web page from the server only if your server is listening at port 80.

Then try to get a file that is not present at the server. You should get a "404 Not Found" message.

What to Hand in

You will hand in the complete server code along with the screen shots of your client browser, verifying that you actually receive the contents of the HTML file from the server.**Skeleton Python Code for the Web Server**

```
#import socket module
from socket import *
import sys # In order to terminate the program
serverSocket = #Fill in start #Fill in end
#Prepare a sever socket
#Fill in start
#Fill in end
while True:
#Establish the connection print('Ready to serve...')
connectionSocket, addr = #Fill in start #Fill in end
try:
message =
#Fill in start #Fill in end
filename = message.split()[1]
f = open(filename[1:])
```

outputdata = **#Fill in start #Fill in end**

Send the HTTP response header line to the

connection socket

#Fill in start #Fill in end

#Send the content of the requested file to the

client

for i in range(0, len(outputdata)):

connectionSocket.send(outputdata[i].encode())

connectionSocket.send("\r\n".encode())connectionSocket.close()

except IOError:

#Send response message for file not found

#Fill in start

#Fill in end

#Close client socket

#Fill in start

#Fill in end

serverSocket.close()

sys.exit()#Terminate the program after sending the

corresponding data

Optional Exercises

1.

Currently, the web server handles only one HTTP request at a time. Implement a multithreaded server that is capable of serving multiple requests simultaneously. Using threading, first create a main thread in which your modified server listens for clients at a fixed port. When it receives a TCP connection request from a client, it will set up the TCP connection through another port and services the client request in a separate thread. There will be a separate TCP connection in a separate thread for each request/response pair.

2.

Instead of using a browser, write your own HTTP client to test your server. Your client will connect to the server using a TCP connection, send an HTTP

request to the server, and display the server response as an output. You can

assume that the HTTP request sent is a GET method.

The client should take command line arguments specifying the server IP address or

host name, the port at which the server is listening, and the path at which therequested object is stored at the server. The following is an input command format

to run the client.

client.py server_host server_port filename